Project Synopsis: Online Test Taking Web Application (MERN Stack)

Project Title:
Online Test Taking Web Application

Objective:
The objective of this project is to develop a complete online test-taking platform that enables teachers to create, manage, and evaluate tests, while allowing students to attempt them in real time. The system implements role-based authentication, automated evaluation, and real-time countdown timers, ensuring a secure and efficient digital examination environment.

Problem Statement:
Traditional examination systems involve extensive manual work — from question paper creation to evaluation and result processing. These methods are time-consuming, error-prone, and lack scalability for online use. This project aims to digitize the entire examination lifecycle by offering a unified online platform for teachers and students. Teachers can create tests and monitor results automatically, while students can appear for tests from any location with live timing and instant result evaluation.

Project Scope:
This web-based solution provides a role-specific environment for teachers and students:

- Teachers can create, schedule, edit, and delete tests.
- Students can view and attempt tests online with timer-based submission.
- The application automatically evaluates objective-type questions and displays results instantly.
- Secure authentication and authorization ensure privacy and controlled access for both roles.

Key Highlights:

- Dual Role System: Separate dashboards for teachers and students.
- Automated Scoring: Instant result calculation and grading.
- Real-Time Monitoring: Live countdown timer and automatic submission upon timeout.
- Secure Authentication: Token-based login using JWT.
- Responsive UI: Built using React and Tailwind CSS for modern interface and accessibility.

Tech Stack:

Layer	Technology Used	Description
Frontend	React.js, Tailwind CSS, React Router, Axios	Dynamic UI and client-side routing
Backend	Node.js, Express.js	RESTful API development
Database	MongoDB, Mongoose	Schema-based NoSQL database
Authentication	JWT, bcrypt.js	Secure token-based login and encryption
Validation	express-validator	Input and form data validation
State Management	React Hooks, Context API	Managing user and test data globally

Authentication and Authorization:

- JWT tokens are used for secure, session-based authentication.
- Passwords are encrypted using bcrypt.js before storage.
- Role-based access control ensures that:
    - Teachers can create, edit, and manage tests.
    - Students can only view and attempt assigned tests.
- A middleware (authUser) verifies tokens and roles before granting access to protected endpoints.

System Architecture:
The system follows the MERN architecture (MongoDB, Express.js, React.js, Node.js).
Frontend (React) communicates with the Backend API (Express) which interacts with the Database (MongoDB).

Data Flow:

1. User logs in using email and password.
2. Backend generates a JWT token upon successful login.
3. Token is stored in the client's local storage.
4. All protected routes and API calls require this token in the request header.
5. Middleware validates the token before allowing data access or modifications.

Folder Structure:

```
client/                      # React Frontend
  ├── pages/
  ├── components/
  └── config/axios.js

backend/                     # Express Backend
  ├── controllers/            # Business logic
  ├── models/                 # Mongoose schemas
  ├── routes/                 # API route handlers
  ├── middleware/             # Authentication middlewares
  └── index.js                # Entry point
```

Database Schema Overview:

1. **User Model (usermodel.js)**
    - Stores user details and manages authentication.
    - Fields: name, email, password, role ('student' or 'teacher').
    - Methods:
        - `hashPassword(password)` → Encrypts password using bcrypt.
        - `isValidPassword(password)` → Validates credentials during login.
        - `generateJWT()` → Generates authentication token with role and ID.
2. **Test Model (testmodel.js)**
    - Stores test-related data created by teachers.
    - Fields: teacherId, title, description, subject, duration, startTime, endTime, status, and questions array.
    - Question Schema includes:
        - id, text, type (multiple-choice or numerical), options, correctAnswer, points, tolerance, and optional imageUrl.
3. **Test Attempt Model (testAttempt.js)**
    - Stores student attempts, answers, and calculated scores.
    - Fields: studentId, testId, answers (with selected options or numerical values), score, totalPoints, and completedAt.

API Endpoints Overview:

Teacher Routes (teacher.routes.js)

- `GET /api/teacher/tests` → Fetch all tests created by a teacher.
- `POST /api/teacher/tests` → Create new test with validation.

Student Routes (student.routes.js)

- `GET /api/student/tests` → Fetch available and upcoming tests.
- `GET /api/student/questions/:testId` → Get questions for a specific test.
- `POST /api/student/submit/:testId` → Submit a test attempt.
- `GET /api/student/results/:testId` → View result of completed test.

Result Routes (result.routes.js)

- `GET /api/result/test/:testId` → Teacher view of all student results.
- `GET /api/result/test/:testId/student/:studentId` → View detailed result for a single student.

User Routes (user.routes.js)

- `POST /api/user/register` → Register as student or teacher.
- `POST /api/user/login` → Login and receive JWT token.
- `GET /api/user/profile` → Fetch user profile using token.
- `GET /api/user/logout` → Logout user and invalidate session.
- `GET /api/user/all` → Retrieve list of all users.

Functional Workflow:

For Teachers:

1. Login/Register → JWT issued.
2. Create Test → Add questions, schedule, and duration.
3. Manage Tests → Edit or delete upcoming tests.
4. View Results → Access performance of all students.

For Students:

1. Login/Register → JWT issued.
2. View Available Tests → Access scheduled tests.
3. Attempt Test → Countdown timer begins.
4. Auto Submission → On timeout or manual completion.

5. View Instant Results → Auto-evaluated score displayed.

Core Functionalities:

- Real-time countdown timer.
- Automatic test submission on timeout.
- Auto evaluation of objective-type questions.
- Role-based dashboards and controls.
- Secure authentication and authorization.
- Dynamic test status transitions (Upcoming → Ongoing → Completed).

Middleware:

- `authUser`: Validates JWT and verifies user role.
- `express-validator`: Validates form input to ensure data integrity.

Frontend Routing Overview (React – App.js):
The frontend is implemented using React Router v6, managing navigation between modules and enforcing role-based access control through the `ProtectedRoute` component.

Route Structure:
The entire routing is wrapped within:

```
<AuthProvider>
  <Router>
    <Routes> ... </Routes>
    <Toaster position="top-right" />
  </Router>
</AuthProvider>
```

AuthProvider manages authentication context, Router enables client-side navigation, and Toaster displays user notifications.

Teacher Routes:

- `/teacher` → Displays teacher dashboard (Protected, teacher role).
- `/teacher/create-test` → Allows creation of new tests (Protected, teacher role).
- `/teacher/results/:testId` → Displays student results for a test.
- `/teacher/results/:testId/:studentId` → Shows detailed report for a single student.

Student Routes:

- `/student` → Student dashboard showing available tests.
- `/student/take-test/:testId` → Test-taking interface with timer.
- `/student/results/:testId` → Displays test result.

User Routes:

- `/login` → Login page for both teacher and student.
- `/register` → Registration page to sign up and choose role.

General Routes:

- `/` → Landing page (default home).
- `/about` → About the application and team.

ProtectedRoute Component:
Ensures secure role-based routing.
If user is not authenticated, redirects to login; if role does not match, redirects to home.

Authentication Flow:

1. User logs in and token is stored via AuthContext.
2. ProtectedRoute checks for valid token and role.
3. Unauthorized users are redirected.
4. On logout, context and local storage are cleared.

Context API (AuthProvider):
Maintains global state of user authentication, handles login/logout, and stores JWT in local storage.
Accessible across all components.

Navigation Example:
Teacher Flow: `/login → /teacher → /teacher/create-test → /teacher/results/:testId`
Student Flow: `/login → /student → /student/take-test/:testId → /student/results/:testId`

Routing Summary:

Type	Example Route	Component	Access
Teacher	/teacher/create-test	CreateTest	Protected
Teacher	/teacher/results/:testId	TeacherTestResults	Protected
Student	/student/take-test/:testId	TakeTest	Protected
Student	/student/results/:testId	TestResults	Protected
Auth	/login, /register	Login/Register	Public
General	/about, /	About/Landing	Public

Expected Outcomes:

- A secure, responsive, and user-friendly online examination system.
- Automated test evaluation reduces manual workload.
- Enhanced flexibility for remote learning environments.
- Scalable structure ready for future improvements.

Future Enhancements:

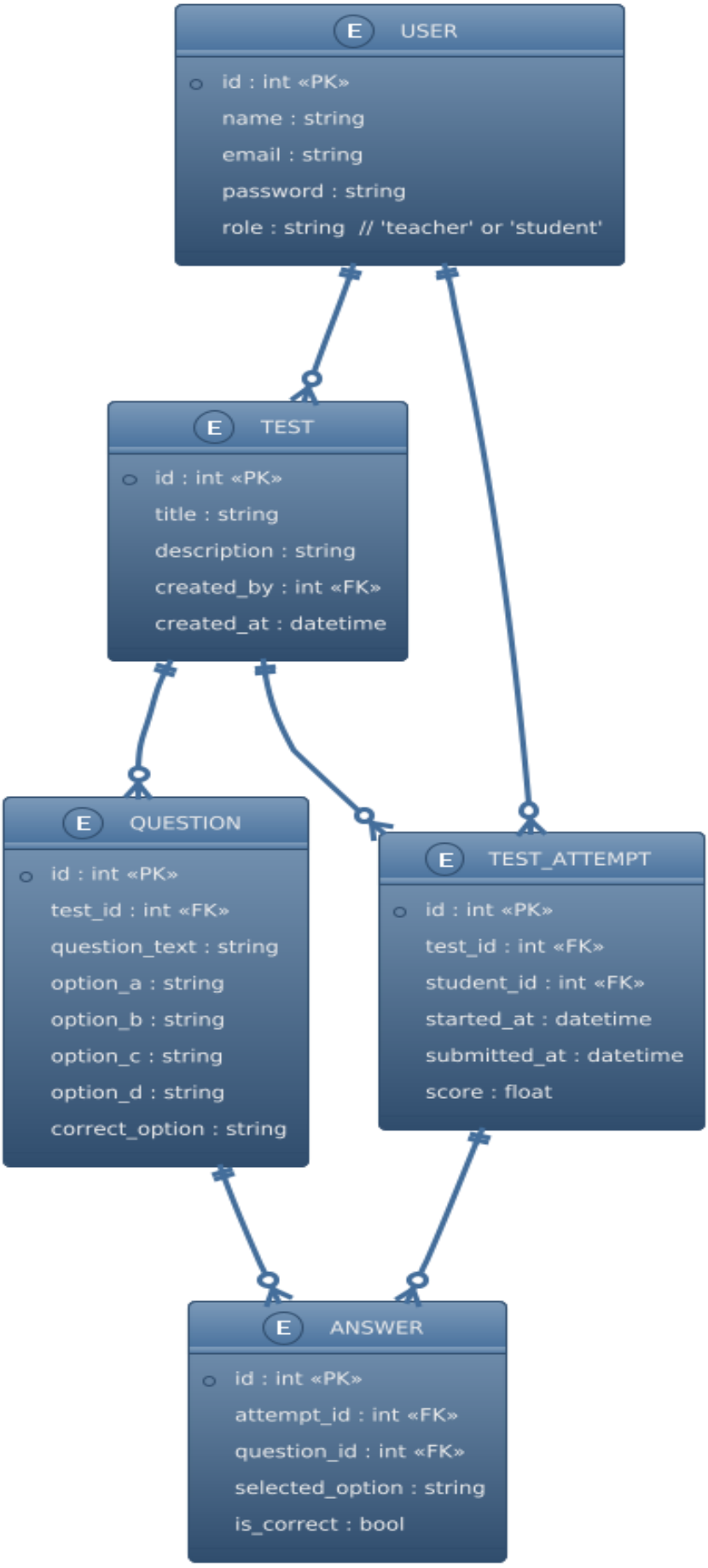
- AI-based subjective answer evaluation.
- Question bank with randomized test generation.
- Advanced analytics and reporting dashboards.
- Email alerts and notifications for upcoming tests.
- Centralized admin management module.

Project Status:

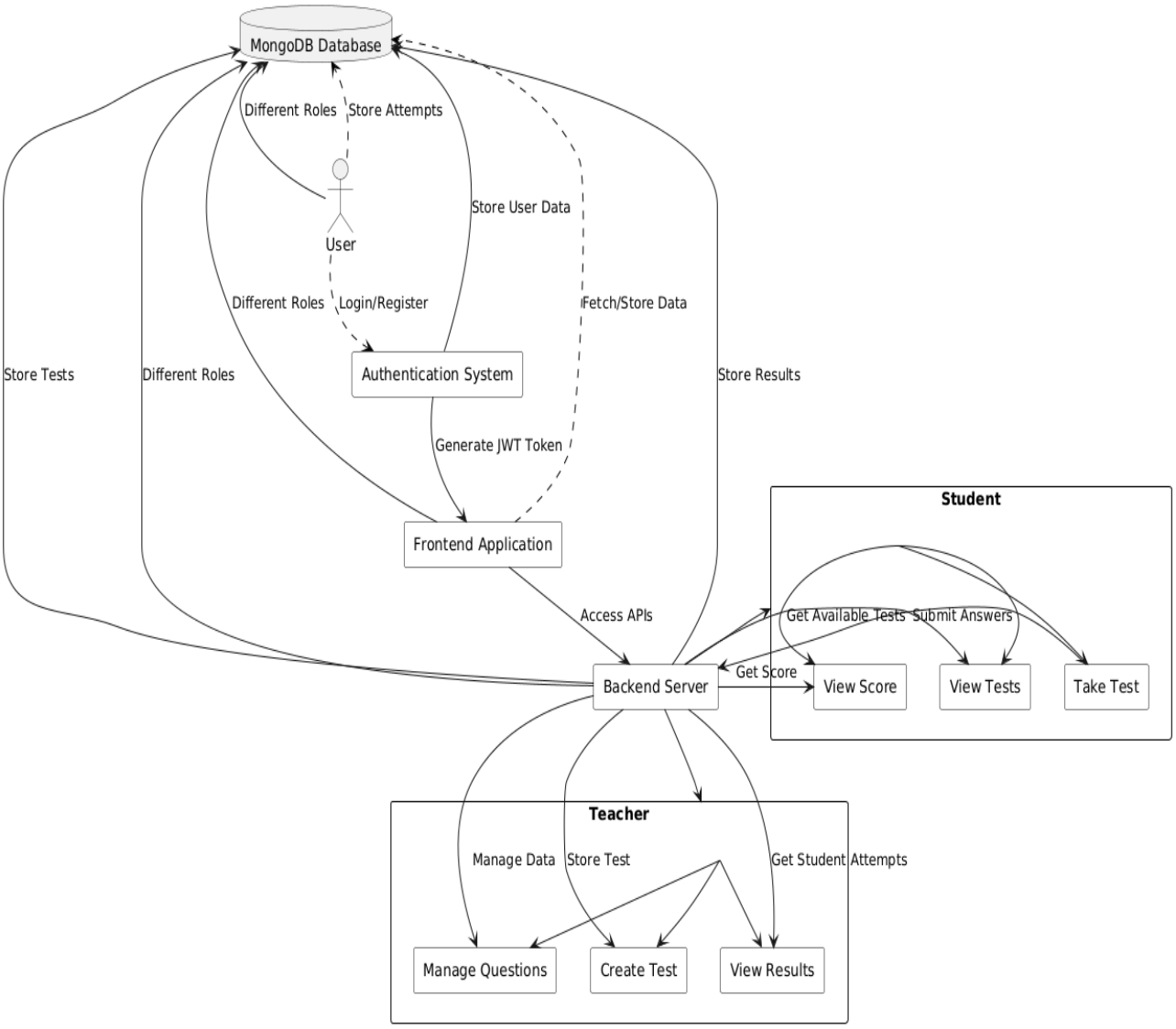
- Frontend Development: Completed (React + Tailwind CSS).
- Backend Development: Completed (Node.js + Express + MongoDB).
- Integration & Testing: In Progress.
- Live Deployment: Coming soon.

Developed By:
Vishwas Chourasiya,
Arun kumar pandey ,
Satyam Pyasi
Full Stack MERN Developer
MCA 3rd Semester – Shri Ram Institute of Technology

ER-Diagram:


USER
id : int «PK»
name : string
email : string
password : string
role : string // 'teacher' or 'student'
TEST
id : int «PK»
title : string
description : string
created_by : int «FK»
created_at : datetime
QUESTION
id : int «PK»
test_id : int «FK»
question_text : string
option_a : string
option_b : string
option_c : string
option_d : string
correct_option : string
TEST_ATTEMPT
id : int «PK»
test_id : int «FK»
student_id : int «FK»
started_at : datetime
submitted_at : datetime
score : float
ANSWER
id : int «PK»
attempt_id : int «FK»
question_id : int «FK»
selected_option : string
is_correct : bool

DFD:


MongoDB Database
Different Roles
Store Attempts
User
Store User Data
Different Roles
Login/Register
Fetch/Store Data
Store Tests
Different Roles
Authentication System
Store Results
Generate JWT Token
Frontend Application
Student
Access APIs
Get Available Tests
Submit Answers
Get Score
Backend Server
View Score
View Tests
Take Test
Teacher
Manage Data
Store Test
Get Student Attempts
Manage Questions
Create Test
View Results